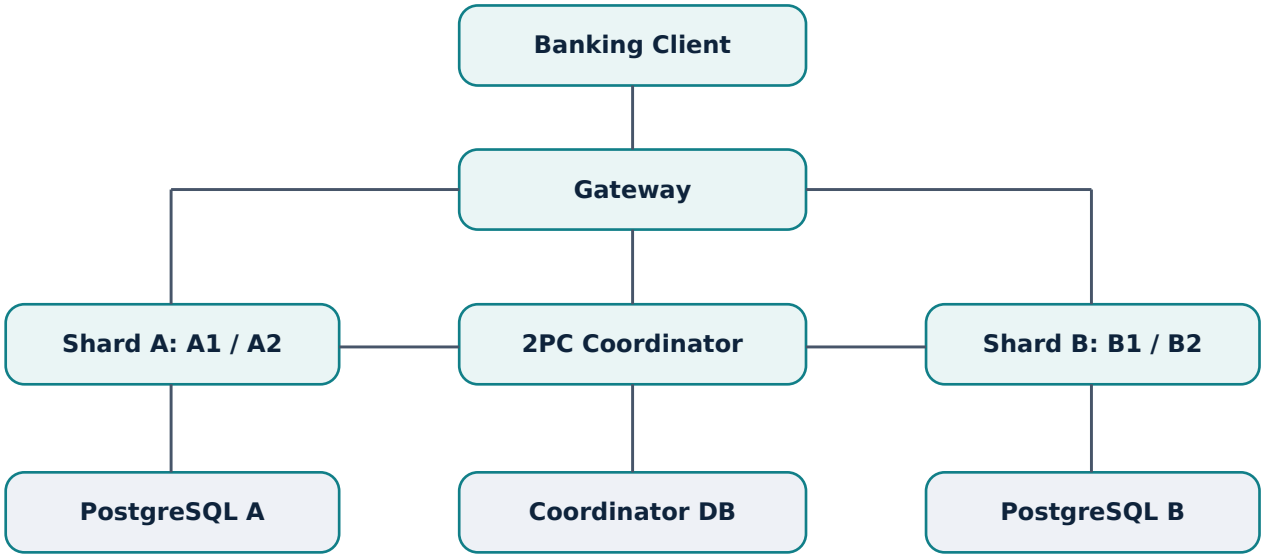
Distributed Banking Platform

Assignment 2 - Sharding, Replication & Distributed Transactions

Core stack: Python 3.12, FastAPI, PostgreSQL 16, psycopg3, Docker Compose.

Scope: Two independent account shards, two stateless application replicas per shard, Gateway failover, local ACID operations, real PostgreSQL 2PC, durable decisions and recovery.



Live verification (2026-10-25): An Apple Silicon Mac with Docker 29.1.3 and Compose 2.40.3 built and ran all nine core containers. Actual student-supplied terminal transcripts confirm all seven experiments, 5/5 protocol tests, real Coordinator/participant recovery, and PostgreSQL restart persistence. A final seven-image rebuild was followed by a second successful run of every experiment and the participant-outage recovery test.

1. Sharding and routing

Shard	Account range	App replicas	Persistent database
A	A0001 - A1000	A1, A2	db-a
B	A1001 - A2000	B1, B2	db-b

A JSON Shard Map defines disjoint numeric ranges. Both Gateway and shard servers reject invalid/misrouted account IDs. Each authoritative balance exists only in its owning PostgreSQL database; both replicas of a shard share that database.

2. Load balancing and availability

The Gateway polls each replica every second, verifies the response shard ID, and uses a per-shard, lock-protected Round Robin counter. Transport failures and HTTP 5xx responses mark that replica unhealthy; subsequent requests are retried on another healthy replica of the **same shard**. Each response and log identifies the replica.

3. Concurrency and local ACID

• Balances use NUMERIC(20,2); negative balance is disallowed by SQL constraints.

• Deposits, withdrawals, and same-shard transfers use database transactions and row-level FOR UPDATE locks; transfer locks both accounts in stable ID order.

• Every local write has a UUID idempotency receipt. Replays return its stored result; conflicting reuse gets HTTP 409. Lock waits have a bounded timeout.

• Seed creates 2,000 sample accounts; subsequent restarts skip an already complete shard seed, avoiding locks from prepared transactions. Docker volumes preserve data and journals.

4. Real two-phase commit (2PC)

Cross-shard transfers are assigned a unique transaction UUID. The Coordinator persistently tracks INITIATED, PREPARING, PREPARED, COMMIT_DECIDED / ABORT_DECIDED, and COMPLETED. Participant-side PREPARE performs a tentative debit or credit plus receipt insertion, then calls PostgreSQL **PREPARE TRANSACTION**. This is not a pair of unrelated committed UPDATE statements.

Step	Coordinator / participant interaction	Durability point
1	Coordinator -> DB C: INSERT INITIATED, then PREPARING	Coordinator journal
2	Coordinator -> A: PREPARE debit; A -> Coordinator: YES	Postgres A prepared XID
3	Coordinator -> B: PREPARE credit; B -> Coordinator: YES	Postgres B prepared XID
4	Coordinator -> DB C: PREPARED, then COMMIT_DECIDED	**Commit decision durable before messages**
5	Coordinator -> A/B: COMMIT; each runs COMMIT PREPARED	Atomic balance + receipt
6	Coordinator -> DB C: store each ACK; COMPLETED	Both ACKs required for success

NO vote: Coordinator records ABORT_DECIDED before messaging both shards. ABORT creates a durable participant tombstone before ROLLBACK PREPARED, preventing a late PREPARE from undoing the abort. Duplicate COMMIT or ABORT is detected by the participant journal, prepared-XID listing and in-transaction committed receipts.

Response rule: COMMITTED is returned only after both participant ACKs. When a participant is unreachable, the externally visible state is PENDING, retrievable via transaction ID or original request UUID.

5. Durable recovery and failure injection

The Coordinator DB stores every global decision and participant acknowledgement. On restart and every two seconds, the recovery worker scans incomplete records. Database advisory locks serialize recovery with active coordinations. Undecided transactions are durably ABORTed; COMMIT_DECIDED transactions are retried toward COMMIT only. The decision is never reversed after persistence.

Injected failure	Expected invariant	Executable proof
First shard YES, second NO	Abort first prepared XID, no balance effect	test_protocol.py: second vote rejection
Coordinator stops after PREPARED	After restart, durable ABORT and rollback both	experiment_6.sh: after_prepare
Coordinator stops after COMMIT_DECIDED	Recover same COMMIT, collect both ACKs	experiment_6.sh: after_commit_decision
Shard B unreachable after COMMIT	PENDING until restored, then COMMITTED	experiment_participant_disconnect.sh
Repeated COMMIT/ABORT	No double debit/credit; receipts unchanged	test_duplicate_commit_messages
Concurrent overlapping transfers	Atomic outcomes and conserved sum	test_experiment_7

A hard Coordinator crash uses **os._exit(91)** immediately after a durable PREPARED or COMMIT_DECIDED write. The restart test queries DB C, checks the retained decision, then verifies participant receipts, cleanup of pg_prepared_xacts and final acknowledgements. Fault injection is opt-in and internal-token protected.

6. Required experiments and observed status

#	Experiment	Acceptance check	Mac result
1	Shard distribution	Exclusive DB records + correct routing	PASS
2	Replica load	A1/A2 and B1/B2 observed	PASS
3	Concurrent replica operations	40 writes; balance 1000.00	PASS
4	Replica interruption	Stop A1; 183 served, 0 failed	PASS
5	Local and distributed transfer	Atomic local SQL + both 2PC ACKs	PASS
6	Interrupted 2PC	PREPARED -> ABORT; COMMIT_DECIDED -> COMMIT	PASS
7	Concurrent shared-account 2PC	12 committed; sum 4000.00	PASS

Supplementary observed checks: Five protocol tests passed (retry-safe deposit/transfer, participant NO vote, repeated COMMIT, insufficient funds). Participant B outage: durable COMMIT remained PENDING with A acknowledged; B returned; both ACKs became true and the same transaction finished COMMITTED. The outage test passed again after rebuilding all images. Both shard DBs showed zero remaining pg_prepared_xacts; all three PostgreSQL servers reported max_prepared_transactions=100.

Persistence: After actual restarts of db-a, db-b, and db-coordinator, balances A0091=979.00 and A1091=1021.00 were unchanged. An existing Coordinator record remained COMPLETED / COMMIT with both acknowledgements true. The full non-disruptive integration suite passed 10/10 tests in 2.96 seconds without cache warnings; 4 unit tests and a full CLI smoke test also passed on the rebuilt images.

7. Limitations and next steps

• Classic 2PC may block participant rows while the Coordinator or a DB is unreachable. The assignment does not require PostgreSQL database failover.

• Public end-user authentication, TLS, audit compliance, multicurrency, high availability for Coordinator DB, and production observability are outside this educational core.

• Optional Bonus I (live migration to shard C with CDC, epochs and crash recovery) and Bonus II (custom automatic replica replacement) are NOT IMPLEMENTED and require prior student approval.

• Mac validation transcripts constitute actual external Docker evidence; the original code-generation sandbox had no Docker. GitHub/Docker Hub publication is not done.

• A fault test revealed redundant account seeding could block on prepared XIDs; the startup guard and no-deps Coordinator restart were applied and recovery retested.

8. Artifacts / reproducibility

• docker-compose.yml, Dockerfile, .env.example and PostgreSQL SQL schemas;

• Four FastAPI banking replicas, shard-aware Gateway, independent 2PC Coordinator, CLI;

• Unit tests, DB-backed integration tests, seven experiment scripts plus participant-outage fault injection;

• README, architecture/2PC sequence diagram, requirement matrix, 15-minute demo plan, GitHub and Docker Hub instructions.

Engineering conclusion. The mandatory 2PC protocol and seven graded experiments were exercised successfully on the student's Docker host, including real prepared XIDs, Coordinator hard crashes, participant disconnection, replay and persistence. The observed startup deadlock was repaired and retested. This is an educational prototype, not a production-ready financial system; publish only after replacing development secrets.

Reference: University PDF, Assignment 2: Distributed Banking Platform, sections 1-9. The repository README gives complete commands and reproducibility details.